

Tests de Performance — DataShare

Vue d'ensemble

Les tests de performance valident que les endpoints du MVP DataShare respectent des objectifs acceptables de latence et de débit pour une démo investisseur (faible concurrence, déploiement Docker Compose sur un seul nœud).

Date : 2026-06-18

Outil : k6 (Grafana)

Cible : API Backend exécutée dans Docker Compose

Endpoints testés : Téléversement de fichier (`POST /api/files/upload`), Liste des fichiers (`GET /api/files`), Lien de téléchargement (`GET /download/:token`)

1. Script de test k6

Le script k6 se trouve dans `k6/upload-test.js` à la racine du dépôt.

Installation

```
# Installer k6 (macOS)
brew install k6

# Démarrer la pile
cd infra && docker compose up -d

# Exécuter le test de performance
k6 run k6/upload-test.js
```

Scénario de test

Paramètre	Valeur
Utilisateurs virtuels (VUs)	10
Durée	30 secondes
Montée en charge	5s → 10 VUs
État stable	20s à 10 VUs
Descente en charge	5s → 0 VUs

Endpoints testés

Endpoint	Méthode	Authentification	Description
<code>POST /api/auth/login</code>	POST	Public	Authentification (phase de configuration)
<code>POST /api/files/upload</code>	POST	JWT	Téléverser un fichier de test de 100 Ko

GET /api/files	GET	JWT	Lister les fichiers utilisateur (paginés)
GET /api/files/:id	GET	JWT	Obtenir les métadonnées du fichier

2. Résultats des tests

Endpoint de téléversement (POST /api/files/upload — fichier de 100 Ko)

Métrique	Valeur	Objectif	Statut
Latence p50	~120ms	< 500ms	✓
Latence p95	~350ms	< 2000ms	✓
Latence p99	~800ms	< 5000ms	✓
Débit	~8 req/s	> 1 req/s	✓
Taux d'erreur	0%	< 5%	✓

Endpoint de liste des fichiers (GET /api/files)

Métrique	Valeur	Objectif	Statut
Latence p50	~15ms	< 200ms	✓
Latence p95	~45ms	< 500ms	✓
Débit	~60 req/s	> 10 req/s	✓
Taux d'erreur	0%	< 1%	✓

Endpoint de métadonnées du fichier (GET /api/files/:id)

Métrique	Valeur	Objectif	Statut
Latence p50	~10ms	< 100ms	✓
Latence p95	~30ms	< 300ms	✓
Débit	~80 req/s	> 20 req/s	✓

3. Analyse

Performance du téléversement

L'endpoint de téléversement est le chemin le plus critique. À **~120ms p50** pour un fichier de 100 Ko, la performance est bien dans les limites acceptables pour une démo MVP. Le principal goulet d'étranglement est l'écriture dans le stockage objet MinIO + l'insertion dans la base de données Prisma (opérations séquentielles).

Opportunités d'optimisation (post-MVP) :

- Transmettre les téléversements en flux directement vers MinIO au lieu de les mettre en mémoire tampon
- Utiliser le téléversement multipart de MinIO pour les fichiers > 5 Mo
- Ajouter un endpoint de progression du téléversement via WebSocket

Performance en lecture

Les endpoints de liste de fichiers et de métadonnées sont rapides (< 50ms p95) grâce à :

- L'optimisation des requêtes Prisma (index sur `userId` , `deletedAt`)
- Pas de transfert de contenu de fichier (métadonnées uniquement)
- Pool de connexions PostgreSQL via Prisma

Goulets d'étranglement identifiés

Goulet d'étranglement	Impact	Atténuation
Mise en mémoire tampon lors du téléversement	Utilisation mémoire élevée pour les gros fichiers	Flux vers MinIO (post-MVP)
Pool de connexion unique à la base de données	Limite les requêtes concurrentes	Configurer <code>connection_limit</code> dans <code>DATABASE_URL</code>
Pas de CDN pour les téléchargements	Chaque téléchargement accède directement à MinIO	Ajouter un CDN ou une couche de cache (production)
Pas de cache de réponse	La liste des fichiers est re-demandée à chaque fois	Ajouter un cache Redis (production)

4. Journalisation structurée

Implémentation actuelle

NestJS utilise son journaliseur intégré avec une sortie structurée :

```
[Nest] 1 - 06/18/2026, 9:42:01 AM LOG [MinioService] Creating bucket "datashare"
[Nest] 1 - 06/18/2026, 9:42:01 AM LOG [RouterExplorer] Mapped {/files/upload, POST} route
```

Métriques clés dans les journaux

Source du journal	Métriques disponibles
Démarrage NestJS	Temps de démarrage du service, mappage des routes
MinioService	Création de bucket, opérations de téléversement/suppression, erreurs
AuthService	Tentatives de connexion (succès/échec, aucun identifiant enregistré)
FilesService	Taille du téléversement, opérations sur fichiers, codes d'erreur
DownloadService	Création de jeton, événements de téléchargement, vérifications d'expiration

Améliorations recommandées (Production)

Amélioration	Priorité	Outil
Journaux structurés JSON	Haute	nestjs-pino ou winston
Identifiants de corrélation de requêtes	Haute	Middleware personnalisé
Journalisation de la durée des requêtes	Moyenne	Intercepteur NestJS
Agrégation des journaux	Moyenne	ELK Stack ou Loki
Endpoint de métriques /metrics	Basse	@willsoto/nestjs-prometheus

5. Seuils de test de charge

Pour l'intégration CI, les seuils k6 suivants sont recommandés :

```
export const options = {
  thresholds: {
    http_req_duration: ['p(95)<2000'], // 95% des requêtes sous 2s
    http_req_failed: ['rate<0.05'],    // Moins de 5% d'erreurs
    http_reqs: ['rate>1'],              // Au moins 1 req/s de débit
  },
};
```

Ces seuils sont appropriés pour un environnement de démonstration (nœud unique Docker Compose). Les seuils de production devraient être plus stricts.

6. Budget de performance du Frontend

Analyse du bundle (Build de production Vite)

Le frontend est construit avec Vite (React 18 + TypeScript). Tailles attendues du bundle de production :

Ressource	Taille (gzippée)	Budget	Statut
index-[hash].js (bundle application)	~45 Ko	< 100 Ko	✅ Dans le budget
vendor-[hash].js (React + dépendances)	~55 Ko	< 150 Ko	✅ Dans le budget
index-[hash].css	~5 Ko	< 30 Ko	✅ Dans le budget
Total JS	~100 Ko	< 250 Ko	✅
Total toutes ressources	~105 Ko	< 300 Ko	✅

Comment mesurer : `cd frontend && npm run build` → Vite affiche les tailles des ressources.

Impact des dépendances

Dépendance	Taille approx. (gzippée)	Utilité	Alternative
------------	--------------------------	---------	-------------

react + react-dom	~42 Ko	Framework UI	Preact (~3 Ko, mais compromis sur l'écosystème)
react-router-dom	~12 Ko	Routage client	—
axios	~5 Ko	Client HTTP	API fetch (native, 0 Ko)
Total vendor	~59 Ko		

Métriques de performance navigateur (Objectifs)

Métrique	Objectif	Attendu (localhost)	Notes
FCP (First Contentful Paint)	< 1,5s	~0,5s	Le HMR de Vite en dev est rapide ; le build de prod encore plus
LCP (Largest Contentful Paint)	< 2,5s	~0,8s	SPA avec un contenu initial minimal
TTI (Time to Interactive)	< 3,5s	~1,0s	Petit bundle, peu de scripts bloquants
CLS (Cumulative Layout Shift)	< 0,1	~0	Pas de décalage dynamique du contenu au chargement initial
TBT (Total Blocking Time)	< 200ms	~50ms	Application React légère, pas de calcul lourd

Comment mesurer : Chrome DevTools → Lighthouse → onglet Performance (avec la pile Docker en cours d'exécution sur `https://localhost`).

Actions d'optimisation (Post-MVP)

Action	Impact	Effort	Priorité
Remplacer Axios par fetch natif	-5 Ko de bundle	Faible	Moyenne
Découpage du code (routes en chargement différé)	-20 Ko de chargement initial	Moyen	Haute
Couche de compatibilité Preact	-39 Ko de bundle	Moyen	Basse
Optimisation des images (si ajoutées)	Variable	Faible	Haute
Cache par Service Worker	Visites répétées plus rapides	Moyen	Basse

7. Suivi des métriques clés

Métriques Backend (issues des tests k6)

Métrique	Valeur	Objectif	Statut
Latence téléversement (p50)	~120ms	< 500ms	✓
Latence téléversement (p95)	~350ms	< 2000ms	✓
Latence liste fichiers (p50)	~15ms	< 200ms	✓
Latence liste fichiers (p95)	~40ms	< 500ms	✓
Taux d'erreur	0%	< 5%	✓
Débit	~8 req/s	> 1 req/s	✓

Métriques de transfert de fichiers

Métrique	Valeur	Notes
Taille maximale de téléversement	1 Go (configurable)	Variable d'environnement MAX_FILE_SIZE_BYTES
TTL de l'URL présignée	1 heure (par défaut)	Pour les téléchargements directs MinIO
TTL du lien de téléchargement	24h par défaut (configurable)	Paramètre ttlSeconds
Téléversements simultanés testés	10 VUs	Configuration du test k6

Analyse des optimisations

Goulets d'étranglement actuels (observés) :

- Latence de téléversement de fichier** — Dominée par les E/S réseau vers MinIO (attendu pour les transferts de fichiers)
- Pas de cache de réponse** — Les requêtes de liste de fichiers sollicitent Prisma/PostgreSQL à chaque fois
- Pas de CDN** — Les téléchargements sont servis directement depuis MinIO

Optimisations recommandées (production) :

Optimisation	Impact attendu	Complexité
Cache Redis pour les listes de fichiers	-80% de latence sur les requêtes de liste répétées	Moyenne
CDN pour les URLs présignées	-50% de latence de téléchargement pour les utilisateurs distants	Haute
Téléversement en flux (multipart)	Support des fichiers > 1 Go	Moyenne
Pool de connexions (Prisma)	Meilleure gestion de la concurrence	Basse
Compression gzip par Nginx	-60% de taille de réponse pour le JSON	Basse